Augmenting LLM Code Translation with Compiler Analysis for C to Triton Kernel Generation

Xiao Qin
University of Leeds
Leeds, United Kingdom
ppnh4382@leeds.ac.uk

Chunwei Xia
University of Leeds
Leeds, United Kingdom

Zheng Wang
University of Leeds
Leeds, United Kingdom

ABSTRACT

Emerging programming models like Triton enable developers to better exploit modern accelerators, but translating legacy code to Triton remains challenging. While Large Language Models (LLMs) show promise for code translation, they often generate incorrect or suboptimal implementations due to a lack of precise parallelization reasoning. We present TritonPilot, a compiler-assisted LLM framework for translating C loop nests to Triton kernels. TritonPilot extracts parallelization properties using compiler-based analysis, and encodes them as structured constraints and guidance in the LLM prompt, enabling correct and performance-aware parallel code synthesis. Across PolyBench/C and TSVC kernels, TritonPilot achieves near-perfect correctness (100% and 99%), improving over LLM-only baselines. On TSVC at realistic data sizes, the generated Triton kernels achieve 1.02× median performance vs. OpenMP GPU offloading compiled by NVIDIA's production compiler on the same GPU, demonstrating that LLM-generated code can match compiler quality. We further demonstrate generalization on Rodinia and ECP proxy applications. These results show that integrating compiler analysis into LLM-based translation enables both correct and high-performance parallel code generation.

CCS CONCEPTS

• **Software and its engineering** → **Compilers**; • **Computing methodologies** → *Parallel computing methodologies*.

KEYWORDS

LLM code generation, GPU kernels, Triton, polyhedral analysis

ACM Reference Format:
Xiao Qin, Chunwei Xia, and Zheng Wang. 2026. Augmenting LLM Code Translation with Compiler Analysis for C to Triton Kernel Generation. In *Proceedings of Workshop on AI for High Performance Compiler Construction (AI4HPCC '26).* ACM, New York, NY, USA, **??** pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Accelerators such as GPUs are now essential for high-performance scientific computing, yet much existing software is still written in legacy C code that is not designed for parallel execution on modern accelerators. Translating these legacy programs to accelerator-friendly implementations is challenging, as it requires identifying parallelism, restructuring computation, and managing synchronization, tasks that require expertise in programming models and hardware.

Emerging programming models such as Triton [**?**] simplify GPU programming by providing higher-level abstractions for writing efficient kernels. However, mapping legacy C code to Triton still requires careful reasoning about loop-level parallelism, data dependencies, and execution order. Developers must determine which loop dimensions can be safely parallelized, how memory accesses should be organized, and when synchronization is required - decisions that are subtle and error-prone in practice.

Large Language Models (LLMs) have recently shown promise in code translation, including generating GPU kernels from high-level descriptions [**?** **?**]. However, LLMs lack precise reasoning about program semantics, especially data dependencies. As a result, they often produce incorrect or inefficient code, including unsafe parallelization that leads to data races, suboptimal kernel structures, and missing synchronization.

In this paper, we present TritonPilot, a system that augments LLM-based translation with compiler analysis. TritonPilot extracts parallelization properties from C loop nests using polyhedral analysis (PET) [**?**] and LLVM dependence analysis [**?**], and encodes them as structured guidance in the LLM prompt. This allows the model to generate correct and performance-aware Triton code without requiring it to perform dependence analysis itself.

We evaluate TritonPilot on a diverse set of benchmarks and application kernels on an NVIDIA RTX 3090 GPU. Across PolyBench/C and TSVC loop kernels, TritonPilot achieves near-perfect correctness (100% and 99%), significantly improving over LLM-only baselines. At realistic data sizes, the generated Triton kernels match the performance of OpenMP GPU offloading compiled by NVIDIA's `nvc` compiler (1.02× median on the same GPU). We further demonstrate generalization on kernels from Rodinia and ECP proxy applications. This paper makes the following contributions:

- Aa compiler-assisted LLM framework for translating legacy C loop nests to Triton, combining compiler-based analysis with structured prompt guidance.
- A method for encoding compiler-derived parallelization constraints as structured inputs to LLMs, enabling correct and performance-aware code generation.
- Empirical results show that integrating compiler analysis with LLMs improves both correctness and performance.

AI4HPCC '26, July 6–9, 2026, Belfast, Northern Ireland, UK
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM. . . $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 OUR APPROACH

TritonPilot is implemented in approximately 14K lines of Python, using PET/isl for polyhedral analysis and LLVM 17 for fallback dependence analysis. Figure **??** depicts the workflow of TritonPilot using `gemm` as a running example. Given the source program (i.e., a C source file in this work), TritonPilot automatically identifies optimisation candidates and translates them into the target language (i.e., Triton kernels in this work). The TritonPilot pipeline consists of five stages: loop nest extraction, static analysis, prompt construction, LLM-based generation, and iterative refinement, described as follows.

2.1 Loop Nest Extraction

The current implementation of TritonPilot targets loop-level code translation. It leverages PET's Static Control Part (SCoP) detection [**?**] to identify affine loop nests suitable for transformation. It scans each function and selects regions where loop bounds and memory accesses are affine in loop iterators and parameters, requiring no manual annotations. For example, for `gemm` in Figure **??** , PET extracts a SCoP covering the full loop nest and produces a polyhedral representation including iteration domains, access relations, and loop schedules. This representation provides a precise and structured basis for subsequent dependence analysis.

2.2 Static Analysis

TritonPilot performs dependence-driven analysis to derive constraints that govern correct parallelization and code generation. It first identifies parallelizable loop dimensions with explicit validity conditions. In `gemm`, both `i` and `j` are parallelizable since updates to `C[i][j]` do not introduce cross-iteration conflicts. It then computes write-after-read (WAR) dependences using exact polyhedral analysis to determine whether in-place updates are safe. For `gemm`, no WAR dependences are present. Finally, it detects multi-phase computation patterns. In this case, beta scaling and matrix multiplication form two phases that share the array `C`, which affects kernel structuring. When polyhedral analysis is not applicable (e.g., due to non-affine accesses), TritonPilot falls back to LLVM's `DependenceAnalysis` to conservatively infer dependence-carrying loops.

2.3 Prompt Construction

TritonPilot translates analysis results into structured guidance that constrains LLM generation. This step bridges compiler analysis and neural code synthesis. The guidance encodes: (i) valid and invalid parallelization choices with explicit mapping to execution grids, (ii) dependence-aware update strategies (in-place, clone, or sequential), (iii) phase decomposition for correctness, (iv) reduction patterns mapped to Triton primitives (e.g., `tl.sum()`, `tl.dot()`), and (v) required synchronization for cross-iteration or cross-phase dependences. This structured representation reduces ambiguity in the prompt and steers the LLM toward semantically correct and performant implementations.

2.4 Generation and Iterative Refinement

Given the constructed prompt, TritonPilot invokes the LLM (Claude Sonnet 4 [**?**] in this work) to generate the target code. The generated kernel is validated against the source program (compiled by LLVM in our case) using identical test inputs and FP32 tolerance checks. On validation failure, TritonPilot classifies the error and issues a targeted retry prompt. This process repeats for up to a fixed number of attempts (10 in this work), with a context reset after the sixth attempt to mitigate error accumulation.

3 EVALUATION SETUP

Platforms. All experiments are conducted on a GPU workstation equipped with an NVIDIA RTX 3090 GPU (24 GB memory, 936 GB/s memory bandwidth) and a dual-socket Intel Xeon Gold 5218R CPU (80 logical cores, ∼140 GB/s memory bandwidth).

We use PyTorch 2.5.1, Triton 3.1.0, CUDA 12.4, and GCC 11.4.0. We use Claude Sonnet 4 [**?**] as the underlying LLM for code translation.

Benchmarks. We evaluate on three benchmark suites: (1) *PolyBench/C* [**?**]: 30 numerical kernels (stencils, linear algebra, data mining) at default sizes (N=60–120) and 8× scale (N=480–960); (2) *TSVC* [**?**]: 151 loop kernels covering linear traversals, reductions, indirect addressing, and loop-carried dependences, benchmarked at 32 MB and 3200 MB per array (N=8M and 800M); (3) *Application kernels*: 8 kernels from Rodinia (SRAD, lud, gaussian, lavaMD, hotspot, pathfinder) and ECP proxy applications (miniMD LJ force, miniFE SpMV).

Baselines. The primary comparison is against *OMP GPU*—OpenMP GPU offloading compiled by NVIDIA `nvc` with managed memory, running on the *same* GPU as Triton. This is the strongest baseline: it isolates the quality of LLM-generated Triton code from hardware differences, directly comparing two GPU programming models on identical hardware. We additionally report results against *OMP CPU* (80-thread OpenMP on CPU) to quantify the GPU's architectural advantage, and *ST* (single-threaded GCC `-O3`) as a conventional reference point.

Evaluation methodology. All speedups measure kernel-only execution time with data pre-staged on the GPU, averaged over 10–50 iterations after warmup. For the correctness ablation, we compare two LLM configurations: *with analysis* (WA), where the prompt includes compiler-derived guidance, and *no analysis* (NA), where the LLM receives only the C source code. This isolates the contribution of static analysis to generation quality.

4 EXPERIMENTAL RESULTS

4.1 Polybench/C (30 Kernels)

Table **??** summarizes results at standard (1×) problem sizes. All 30 kernels pass with analysis guidance; without it, two

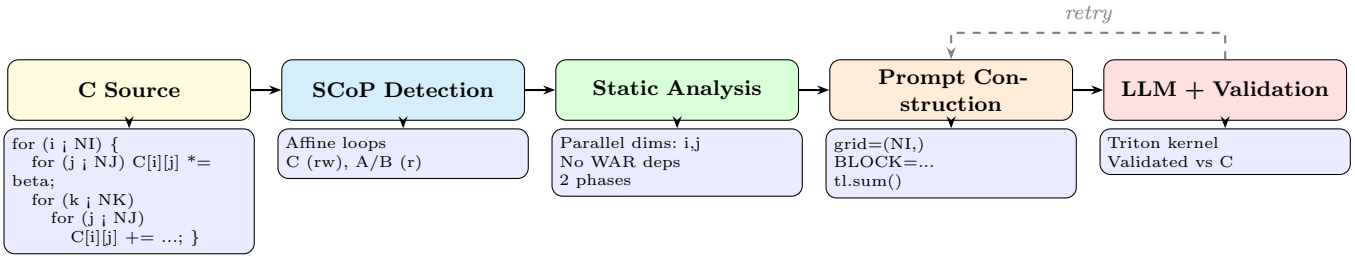

Figure 1: The TritonPilot workflow.

Table 1: Polybench/C results. WA = with analysis, NA = no analysis.

Metric	WA	NA
Pass rate	30/30 (100%)	28/30 (93%)
Median speedup	2.15×	1.06×
Mean speedup	3.44×	1.52×
Kernels > 1×	20/30	14/28
Head-to-head (28 common):		
WA wins (>10% faster)	12	
NA wins (>10% faster)	6 (3 race conditions)	
Ties (within 10%)	10	

Table 2: Polybench scaling: 1× to 8× problem sizes (N: 60→480, matrices: 120→960). "ST" = single-threaded C; "OMP" = 80-thread OpenMP.

Metric	1×	8× (ST)	8× (OMP)
Pass rate	30/30	30/30	30/30
Benchmarked	30	25*	25*
Median speedup	2.15×	30.73×	8.57×
Kernels > 1×	20/30	21/25	20/25

*5 sequential kernels timeout during benchmark.

stencil kernels fail due to parallelizing dimensions with carried dependences. The 2.15× median speedup is limited by Polybench's small defaults ($N = 60$–120), where GPU launch overhead dominates. Compute-bound kernels show strong speedups (`heat_3d`: 12.36×, `deriche`: 7.44×, `covariance`: 7.40×).

Of 6 apparent NA wins, 3 (`lu`, `ludcmp`, `bicg`) contain confirmed race conditions—the NA code uses `grid=(N,)` for LU decomposition where row i reads from row k ($k < i$) that may not yet be computed by its CTA, producing up to 10^{33} variation across runs. The remaining 3 NA wins reflect nondeterministic LLM code quality, not structural differences.

Scaling. At 8× sizes (Table **??**), median speedup rises to 30.73× over single-threaded C. Against 80-thread OpenMP, the GPU still achieves 8.57× median, confirming genuine advantage beyond CPU parallelism. Five sequential kernels time out during benchmark (`grid=(1,)`).

Table 3: Impact of analysis-triggered guidance categories. "Before" and "After" are speedups at 1× sizes except where noted.

Guidance	Kernel	Before	After
Multi-phase split	correlation	0.19×	2.50×
WAR sequential	nussinov	FAIL	PASS
Triangular grid	lud†	0.14×	13.18×
Fused reductions	bicg	1.04×	3.51×
Stencil barriers	jacobi_1d	0.13×	1.78×
IIR phase fusion	deriche	0.28×	7.19×
Tridiag. sweep	adi	0.24×	2.27×

†Rodinia benchmark.

Table 4: TSVC results at three data sizes. "ST" = single-threaded C (`-O2`), "OMP" = OpenMP C (`-O2 -fopenmp`, 80 threads). Pass rate: 150/151 (99.3%).

	32 MB	3200 MB
Per-array size	32 MB	3.2 GB
Parallelizable*	27/150	15/150
Tri/ST median	67.1×	73.2×
Tri/OMP CPU median	10.7×	14.2×
Tri/OMP GPU median	1.02×	—

*Serial/2D kernels excluded at large sizes.

Guidance Impact. Table **??** shows the impact of individual guidance categories, each triggered by analysis properties (e.g., cross-phase dependences, triangular patterns). The largest gains come from multi-phase decomposition (correlation: 13×) and triangular grid (lud: 94×).

Limitations. Ten kernels achieve <1× at standard sizes: five are inherently sequential (loop-carried dependences limit parallelism), and two (`jacobi_2d`, `fdtd_2d`) suffer from kernel launch overhead at small N—at 8× sizes they reach 36.82× and 15.43×.

4.2 TSVC (151 Kernels)

We apply TritonPilot to 151 TSVC [**?**] kernels covering linear traversals, reductions, indirect addressing, and loop-carried dependences.

Table 5: Application kernel results (speedup vs. single-threaded C). "—" = NA failed all attempts.

Source	Kernel	WA	NA
Rodinia	SRAD	24.86×	3.50×
Rodinia	lud	13.18×	15.52×
Rodinia	gaussian	3.35×	3.61×
Rodinia	lavaMD	1.47×	0.56×
Rodinia	hotspot	1.05×	1.22×
Rodinia	pathfinder	0.11×	—
miniMD	LJ force	25.82×	23.00×
miniFE	SpMV	8.95×	7.27×

Table **??** shows results at realistic data sizes. TritonPilot passes 150/151 (99.3%), 120 on first attempt. At 32 MB, LLM-generated Triton achieves 67× (ST) and 10.7× (80-thread OMP CPU). Crucially, Triton achieves **1.02× vs. OpenMP GPU offloading** (`nvc -mp=gpu`) on the *same* GPU, showing the LLM produces code matching a production compiler.

4.3 Application Kernels

To evaluate generalization beyond synthetic benchmarks, we apply TritonPilot to 6 Rodinia kernels [**?**] and 2 ECP proxy application kernels [**?**]—CSR sparse matrix-vector multiply (SpMV) from miniFE and Lennard-Jones force computation from miniMD (Table **??**). All 8 pass correctness with analysis guidance (6 on the first attempt).

The primary evaluation here is *WA vs. NA*—whether analysis guidance improves LLM-generated GPU code. SRAD has three computational phases; analysis detects cross-phase dependences and recommends kernel splitting, yielding 7.1× improvement over unguided code (WA 24.9× vs. NA 3.5×). lavaMD improves from 0.56× (NA) to 1.47× (WA) as analysis identifies the parallelizable dimension; the NA baseline fails on 4 of 5 attempts. For LJ force, guidance promotes efficient neighbor-list traversal (WA 25.8× vs. NA 23.0×). Note that these speedups are vs. single-threaded C and primarily demonstrate analysis impact on code quality, not absolute GPU advantage; the OMP GPU comparison in §**??** provides the hardware-fair baseline.

5 RELATED WORK

LLM Code Generation for GPUs. Chen et al. [**?**] evaluate ChatGPT on CUDA kernel generation, finding frequent correctness issues including data races and incorrect memory management. KernelBench [**?**] benchmarks LLM-generated Triton/CUDA on 250 tasks, reporting 20–40% functional correctness without guidance. Our work differs fundamentally: rather than relying on the LLM's implicit knowledge of parallelism, we provide explicit compiler-derived constraints, achieving 100% correctness on Polybench/C and 99% on TSVC.

Polyhedral Compilation and Auto-Scheduling. Polyhedral frameworks such as Pluto [**?**] and PPCG [**?**] automatically generate parallel code from affine loop nests. PPCG targets CUDA directly but is limited to affine programs and produces low-level code that lacks Triton's block-level optimizations (automatic memory coalescing, shared memory tiling). TVM [**?**] and Halide [**?**] use auto-scheduling with learned cost models for tensor operations. TritonPilot is complementary: rather than replacing the compiler, it uses polyhedral analysis to extract *what* parallelism is safe, then delegates the *how* to the LLM, which can handle non-affine constructs, irregular access patterns, and high-level Triton abstractions that polyhedral compilers cannot target directly.

6 CONCLUSION

We have presented TritonPilot, a system that combines compiler-based analysis with LLM-based kernel generation for accelerators. By translating parallelization constraints, dependence information, and memory access patterns into structured prompts, TritonPilot enables LLMs to generate correct and efficient Triton kernels from C programs.

TritonPilot achieves near-perfect correctness (30/30 PolyBench/C, 150/151 TSVC) and generates Triton code that matches production compiler quality: on TSVC at 32 MB, LLM-generated Triton achieves 1.02× median vs. OpenMP GPU offloading (`nvc`) on the same GPU. It generalizes to 8 application kernels from Rodinia and ECP proxy apps. Ablation results show that analysis-guided prompting is essential for correctness, preventing failures and exposing latent race conditions in unguided generation. Future work includes extending to non-affine programs and integrating autotuning for parameter selection.

REFERENCES

[1] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proc. MAPL*, 2019.

[2] Y. Chen, S. Huang, D. Zheng, et al. ChatGPT for CUDA programming: Evaluating LLM-generated GPU code. *arXiv preprint arXiv:2404.12000*, 2024.

[3] A. Ouyang, S. Zheng, et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2404.12489*, 2024.

[4] S. Verdoolaege and T. Grosser. Polyhedral extraction tool. In *Proc. IMPACT*, 2012.

[5] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proc. CGO*, 2004.

[6] L.-N. Pouchet. Polybench/C: The polyhedral benchmark suite, version 4.2.1. https://web.cs.ucla.edu/~pouchet/software/polybench/, 2016.

[7] Anthropic. Claude Sonnet 4 model card. https://www.anthropic.com/claude, 2025.

[8] S. Verdoolaege. isl: An integer set library for the polyhedral model. In *Proc. ICMS*, 2010.

[9] T. Chen, T. Moreau, Z. Jiang, et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proc. OSDI*, 2018.

[10] J. Ragan-Kelley, C. Barnes, A. Adams, et al. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proc. PLDI*, 2013.

[11] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proc. PLDI*, 2008.

[12] S. Verdoolaege, J. Carlos Juega, A. Cohen, et al. Polyhedral parallel code generation for CUDA. *ACM TACO*, 9(4), 2013.

[13] S. Che, M. Boyer, J. Meng, et al. Rodinia: A benchmark suite for heterogeneous computing. In *Proc. IISWC*, 2009.

[14] M. A. Heroux, D. W. Doerfler, P. S. Crozier, et al. Improving performance via mini-applications. Sandia National Laboratories Technical Report SAND2009-5574, 2009.

[15] D. Callahan, J. Dongarra, and D. Levine. Vectorizing compilers: A test suite and results. In *Proc. Supercomputing*, 1988.